



Object Oriented Programming Language (JAVA)

Lecture 07

Course Code: CSE 06132133

Credit: 3.0

Chapter 7: A Closer Look at Methods and Classes



Overloading Methods

- In Java, it is possible to define **two or more methods** within the **same class** that share the **same name**, as long as their **parameter** declarations are **different**.
- When this is the case, the **methods** are said to be **overloaded**, and the **process** is referred to as **method overloading**.
- **Method overloading** is one of the ways that Java supports **polymorphism**.
- While **overloaded** methods may have **different return types**, the return type alone is **insufficient** to **distinguish** two versions of a method.

```
1  class OverloadDemo {
2      void test() {
3          System.out.println("No parameters");
4      }
5      void test(int a) {
6          System.out.println("a: " + a);
7      }
8      void test(int a, int b) {
9          System.out.println("a and b: " + a + " " + b);
10     }
11     double test(double a) {
12         System.out.println("double a: " + a);
13         return a*a;
14     }
15 }
```



```
1 class Overload {
2     public static void main(String[] args) {
3         OverloadDemo ob = new OverloadDemo();
4         double result;
5         ob.test();
6         ob.test(10);
7         ob.test(10, 20);
8         result = ob.test(123.25);
9         System.out.println("Result of ob.test(123.25): " + result);
10    }
11 }
```

```

1  class OverloadDemo {
2      void test() {
3          System.out.println("No parameters");
4      }
5      void test(int a, int b) {
6          System.out.println("a and b: " + a + " " + b);
7      }
8      void test(double a) {
9          System.out.println("Inside test(double) a: " + a);
10     }
11 }
12 class Overload {
13     public static void main(String[] args) {
14         OverloadDemo ob = new OverloadDemo();
15         int i = 88;
16         ob.test();
17         ob.test(10, 20);
18         ob.test(i); // this will invoke test(double)
19         ob.test(123.2); // this will invoke test(double)
20     }
21 }

```

When an **overloaded method** is called, Java looks for a **match** between the **arguments** used to call the method and the method's **parameters**. However, this match need not always be **exact**. In some cases, Java's **automatic type conversions** can play a role in **overload resolution**.

Output:

No parameters

a and b: 10 20

Inside test(double) a: 88.0

Inside test(double) a: 123.2



Overloading Constructors

```
1  class Box {
2      double width;
3      double height;
4      double depth;
5      // This is the constructor for Box.
6      Box(double w, double h, double d) {
7          width = w;
8          height = h;
9          depth = d;
10     }
11     // compute and return volume
12     double volume() {
13         return width * height * depth;
14     }
15 }
```

```
1  class Box {
2      double width;
3      double height;
4      double depth;
5      Box(double w, double h, double d) {// constructor used when all dimensions specified
6          width = w;
7          height = h;
8          depth = d;
9      }
10     Box() {// constructor used when no dimensions specified
11         width = -1; // use -1 to indicate
12         height = -1; // an uninitialized
13         depth = -1; // box
14     }
15     Box(double len) {// constructor used when cube is created
16         width = height = depth = len;
17     }
18     double volume() {
19         return width * height * depth;
20     }
21 }
```

```
1  class OverloadCons {
2      public static void main(String[] args) {
3          // create boxes using the various constructors
4          Box mybox1 = new Box(10, 20, 15);
5          Box mybox2 = new Box();
6          Box mycube = new Box(7);
7          double vol;
8          // get volume of first box
9          vol = mybox1.volume();
10         System.out.println("Volume of mybox1 is " + vol);
11         // get volume of second box
12         vol = mybox2.volume();
13         System.out.println("Volume of mybox2 is " + vol);
14         // get volume of cube
15         vol = mycube.volume();
16         System.out.println("Volume of mycube is " + vol);
17     }
18 }
```

Using Objects as Parameters



```
1 class Test {
2     int a, b;
3     Test(int i, int j) {
4         a = i;
5         b = j;
6     }
7     boolean equalTo(Test o) {
8         if(o.a == a && o.b == b) return true;
9         else return false;
10    }
11 }
```

```
1 class PassOb {
2     public static void main(String[] args) {
3         Test ob1 = new Test(100, 22);
4         Test ob2 = new Test(100, 22);
5         Test ob3 = new Test(-1, -1);
6         System.out.println("ob1 == ob2: " + ob1.equals(ob2));
7         System.out.println("ob1 == ob3: " + ob1.equals(ob3));
8     }
9 }
```

Output:

ob1 == ob2: true

ob1 == ob3: false

```
1  class Box {
2      double width;
3      double height;
4      double depth;
5      // Notice this constructor. It takes an object of type Box.
6      Box(Box ob) { // pass object to constructor
7          width = ob.width;
8          height = ob.height;
9          depth = ob.depth;
10     }
11     Box(double w, double h, double d) {// constructor used when all dimensions specified
12         width = w;
13         height = h;
14         depth = d;
15     }
16     ...
17     ...
18     ...
19 }
```

```
1 class OverloadCons2 {
2     public static void main(String[] args) {
3         // create boxes using the various constructors
4         Box mybox1 = new Box(10, 20, 15);
5         Box mybox2 = new Box();
6         Box mycube = new Box(7);
7         Box myclone = new Box(mybox1); // create copy of mybox1
8         double vol;
9         // get volume of first box
10        vol = mybox1.volume();
11        System.out.println("Volume of mybox1 is " + vol);
12        // get volume of second box
13        vol = mybox2.volume();
14        System.out.println("Volume of mybox2 is " + vol);
15        // get volume of cube
16        vol = mycube.volume();
17        System.out.println("Volume of cube is " + vol);
18        // get volume of clone
19        vol = myclone.volume();
20        System.out.println("Volume of clone is " + vol);
21    }
22 }
23
```



A Closer Look at Argument Passing

In general, there are **two ways** that a computer language can **pass** an **argument** to a **subroutine**.

- The **first** way is ***call-by-value***. This approach **copies** the **value** of an **argument** into the formal **parameter** of the **subroutine**. Therefore, **changes** made to the **parameter** of the subroutine have **no effect** on the **argument**.
- The **second** way an argument can be passed is ***call-by-reference***. In this approach, a **reference** to an **argument** (**not the value** of the argument) is **passed** to the **parameter**. Inside the subroutine, this **reference** is used to access the actual argument specified in the call. This means that **changes** made to the **parameter** will **affect** the **argument** used to call the subroutine.



A Closer Look at Argument Passing

- Java uses **call-by-value** to pass all **arguments**, the precise effect **differs** between whether a **primitive type** or a **reference type** is passed.
- When you **pass** a **primitive type** to a method, it is ***passed by value***. Thus, a **copy** of the **argument** is made, and what occurs to the **parameter** that receives the argument has **no effect** outside the method.
- When you **pass** an **object** to a method, the situation changes dramatically, because objects are passed by what is effectively **call-by-reference**. Changes to the object inside the method **do affect** the object used as an argument.

```
1 // Primitive types are passed by value.
2 class Test {
3     void meth(int i, int j) {
4         i *= 2;
5         j /= 2;
6     }
7 }
8 class CallByValue {
9     public static void main(String[] args) {
10         Test ob = new Test();
11         int a = 15, b = 20;
12         System.out.println("a and b before call: " + a + " " + b);
13         ob.meth(a, b);
14         System.out.println("a and b after call: " + a + " " + b);
15     }
16 }
```

```
1  class Test {
2      int a, b;
3      Test(int i, int j) {
4          a = i;
5          b = j;
6      }
7      void meth(Test o) {// pass an object
8          o.a *= 2;
9          o.b /= 2;
10     }
11 }
12 class PassObjRef {
13     public static void main(String[] args) {
14         Test ob = new Test(15, 20);
15         System.out.println("ob.a and ob.b before call: " + ob.a + " " + ob.b);
16         ob.meth(ob);
17         System.out.println("ob.a and ob.b after call: " + ob.a + " " + ob.b);
18     }
19 }
```

```
1 class Test {
2     int a, b;
3     Test(int i, int j) {
4         a = i;
5         b = j;
6     }
7     void meth(Test o) {// pass an object
8         o.a *= 2;
9         o.b /= 2;
10    }
11 }
12 class PassObjRef {
13     public static void main(String[] args) {
14         Test ob = new Test(15, 20);
15         System.out.println("ob.a and ob.b before call: " + ob.a + " " + ob.b);
16         ob.meth(ob);
17         System.out.println("ob.a and ob.b after call: " + ob.a + " " + ob.b);
18     }
19 }
```

Output:

ob.a and ob.b before call: 15 20

ob.a and ob.b after call: 30 10

```
1  class Test {
2      int a;
3      Test(int i) {
4          a = i;
5      }
6      Test incrByTen() {
7          Test temp = new Test(a+10);
8          return temp;
9      }
10 }
11 class RetOb {
12     public static void main(String[] args) {
13         Test ob1 = new Test(2);
14         Test ob2;
15         ob2 = ob1.incrByTen();
16         System.out.println("ob1.a: " + ob1.a);
17         System.out.println("ob2.a: " + ob2.a);
18         ob2 = ob2.incrByTen();
19         System.out.println("ob2.a after second increase: " + ob2.a);
20     }
21 }
```

```
1 class Test {
2     int a;
3     Test(int i) {
4         a = i;
5     }
6     Test incrByTen() {
7         Test temp = new Test(a+10);
8         return temp;
9     }
10 }
11 class RetOb {
12     public static void main(String[] args) {
13         Test ob1 = new Test(2);
14         Test ob2;
15         ob2 = ob1.incrByTen();
16         System.out.println("ob1.a: " + ob1.a);
17         System.out.println("ob2.a: " + ob2.a);
18         ob2 = ob2.incrByTen();
19         System.out.println("ob2.a after second increase: " + ob2.a);
20     }
21 }
```

Output:

ob1.a: 2

ob2.a: 12

ob2.a after second increase: 22



Recursion

Java supports **recursion**. **Recursion** is the process of defining something in terms of **itself**. As it relates to Java programming, **recursion** is the attribute that allows a **method** to **call itself**. A method that **calls itself** is said to be **recursive**.

```
1  class Factorial {  
2      // this is a recursive method  
3      int fact(int n) {  
4          int result;  
5          if(n==1) return 1;  
6          result = fact(n-1) * n;  
7          return result;  
8      }  
9  }
```



Introducing Access Control

- Through **encapsulation**, you can **control** what parts of a program can **access** the **members** of a **class**. By controlling access, you can **prevent misuse**.
- How a **member** can be **accessed** is determined by the **access modifier** attached to its **declaration**.
- Java's access modifiers are **public**, **private**, and **protected**. Java also defines a **default** access level. **protected** applies only when **inheritance** is involved.
- When a member of a class is modified by **public**, then that member can be accessed by **any** other **code**.
- When a member of a class is specified as **private**, then that member can only be accessed by other **members** of its **class**.
- When **no access modifier** is used, then by **default** the member of a class is **public** within its **own package**, but **cannot be accessed outside** of its **package**.

```
1 class Test {
2     int a; // default access
3     public int b; // public access
4     private int c; // private access
5     // methods to access c
6     void setc(int i) { // set c's value
7         c = i;
8     }
9     int getc() { // get c's value
10        return c;
11    }
12 }
13 class AccessTest {
14     public static void main(String[] args) {
15         Test ob = new Test();
16         // These are OK, a and b may be accessed directly
17         ob.a = 10;
18         ob.b = 20;
19         // This is not OK and will cause an error
20         // ob.c = 100; // Error!
21         // You must access c through its methods
22         ob.setc(100); // OK
23         System.out.println("a, b, and c: " + ob.a + " " + ob.b + " " + ob.getc());
24     }
25 }
```

```
1 class Stack {
2     private int[] stck = new int[10];
3     private int tos;
4     Stack() {
5         tos = -1;
6     }
7     void push(int item) {
8         if(tos==9)
9             System.out.println("Stack is full.");
10        else
11            stck[++tos] = item;
12    }
13    int pop() {
14        if(tos < 0) {
15            System.out.println("Stack underflow.");
16            return 0;
17        }
18        else
19            return stck[tos--];
20    }
21 }
```



Understanding static

- Normally, a **class member** must be **accessed** only in **conjunction** with an **object** of its class. However, it is possible to **create** a **member** that can be used by itself, **without reference** to a specific **instance**. To create such a member, precede its declaration with the **keyword static**.
- When a **member** is declared **static**, it can be accessed **before** any **objects** of its class are **created**, and **without reference** to any **object**. You can declare **both methods** and **variables** to be **static**.
- The most common example of a **static** member is **main()**. **main()** is declared as **static** because it must be called before any objects exist.
- **Instance variables** declared as **static** are, essentially, **global** variables. When objects of its class are declared, **no copy** of a static variable is **made**. Instead, **all instances** of the class **share** the **same static variable**.



Understanding static

Methods declared as **static** have several **restrictions**:

- They can **only directly call other static methods** of their class.
- They can **only directly access static variables** of their class.
- They **cannot refer to this or super** in any way. (The keyword **super** relates to **inheritance**)

If you need to do computation in order to **initialize** your **static variables**, you can declare a **static block** that gets **executed exactly once**, when the **class is first loaded**.

Outside of the **class** in which they are defined, **static methods** and **variables** can be used **independently** of any **object**. To do so, you need only specify the **name** of their **class** followed by the **dot operator**.

classname.method()

```
1 class UseStatic {
2     static int a = 3;
3     static int b;
4     static void meth(int x) {
5         System.out.println("x = " + x);
6         System.out.println("a = " + a);
7         System.out.println("b = " + b);
8     }
9     static {
10        System.out.println("Static block initialized.");
11        b = a * 4;
12    }
13    public static void main(String[] args) {
14        meth(42);
15    }
16 }
```

```
1  class StaticDemo {
2      static int a = 42;
3      static int b = 99;
4      static void callme() {
5          System.out.println("a = " + a);
6      }
7  }
8  class StaticByName {
9      public static void main(String[] args) {
10         StaticDemo.callme();
11         System.out.println("b = " + StaticDemo.b);
12     }
13 }
```



Introducing final

```
1 final int FILE_NEW = 1;
2 final int FILE_OPEN = 2;
3 final int FILE_SAVE = 3;
4 final int FILE_SAVEAS = 4;
5 final int FILE_QUIT = 5;
```

- A field can be declared as **final**. Doing so **prevents** its contents from being **modified**, making it, essentially, a **constant**. This means that you **must initialize** a **final field** when it is declared.
- It is a common coding **convention** to choose **all uppercase identifiers** for **final** fields
- In addition to fields, both **method parameters** and **local variables** can be declared **final**.
- Declaring a **parameter final** **prevents** it from being **changed** within the method.
- Declaring a **local variable final** **prevents** it from being assigned a value more than once.
- The **keyword final** can also be applied to **methods**, but its meaning is substantially **different** than when it is applied to variables. (explained in the next chapter)



Arrays Revisited

- An important point can be made about **arrays**: they are **implemented** as **objects**. Because of this, there is a special array attribute that you will want to take advantage of.
- Specifically, the **size** of an **array**—that is, the number of elements that an array can hold is found in its **length instance variable**. All arrays have this variable, and it will always hold the size of the array.
- Keep in mind that the value of **length** has nothing to do with the **number** of elements that are actually **in use**. It only **reflects** the **number** of elements that the array is **designed** to **hold**.



Introducing Nested and Inner Classes

- It is possible to **define a class within another class**; such classes are known as **nested classes**.
- The **scope** of a **nested class** is bounded by the **scope** of its **enclosing class**.
- Thus, if **class B** is defined within **class A**, then **B** does not exist independently of **A**.
- A **nested class** has **access** to the members, including **private members**, of the class in which it is nested.
- However, the **enclosing class** does **not** have access to the **members** of the **nested class**.
- It is also possible to declare a **nested class** that is **local** to a **block**.



Introducing Nested and Inner Classes

- There are **two types** of **nested classes**: **static** and **non-static**.
- A **static nested class** is one that has the static modifier applied. Because it is **static**, it must access the **non-static members** of its enclosing class through an **object**. That is, it **cannot refer** to **non-static members** of its enclosing class **directly**.
- The second type of nested class is the **inner class**.
- An **inner class** is a **non-static nested class**. It has **access** to **all** of the **variables** and **methods** of its **outer class** and may refer to them **directly** in the same way that other non-static members of the **outer class** do.

```
1  class Outer {
2      int outer_x = 100;
3      void test() {
4          Inner inner = new Inner();
5          inner.display();
6      }
7      // this is an inner class
8      class Inner {
9          void display() {
10             System.out.println("display: outer_x = " + outer_x);
11         }
12     }
13 }
14 class InnerClassDemo {
15     public static void main(String[] args) {
16         Outer outer = new Outer();
17         outer.test();
18     }
19 }
```

```
1 class Outer {
2     int outer_x = 100;
3     void test() {
4         Inner inner = new Inner();
5         inner.display();
6     }
7     // this is an inner class
8     class Inner {
9         int y = 10; // y is local to Inner
10        void display() {
11            System.out.println("display: outer_x = " + outer_x);
12        }
13    }
14    void showy() {
15        System.out.println(y); // error, y not known here!
16    }
17 }
18 class InnerClassDemo {
19     public static void main(String[] args) {
20         Outer outer = new Outer();
21         outer.test();
22     }
23 }
```

```
1  class Outer {
2      int outer_x = 100;
3      void test() {
4          for(int i=0; i<10; i++) {
5              class Inner {
6                  void display() {
7                      System.out.println("display: outer_x = " + outer_x);
8                  }
9              }
10             Inner inner = new Inner();
11             inner.display();
12         }
13     }
14 }
15 class InnerClassDemo {
16     public static void main(String[] args) {
17         Outer outer = new Outer();
18         outer.test();
19     }
20 }
```



Exploring the String Class

- The **first** thing to understand about **strings** is that every string you create is actually an **object** of type **String**. Even **string constants** are actually **String objects**. For example, in the statement

```
1 System.out.println("This is a String, too");
```

- the string **"This is a String, too"** is a **String object**.
- The **second** thing to understand about **strings** is that **objects** of type **String** are **immutable**; once a **String** object is **created**, its contents **cannot** be **altered**.



Exploring the String Class

While this may seem like a serious restriction, it is not, for **two reasons**:

- If you need to change a **string**, you can always **create** a **new one** that contains the **modifications**.
- Java defines peer classes of **String**, called **StringBuffer** and **StringBuilder**, which allow strings to be **altered**, so all of the normal string **manipulations** are still available in Java.

Java defines one **operator** for **String** objects: **+**. It is used to **concatenate** two strings.

The **String** class contains several **methods** that you can use. Here are a few. You can **test** two strings for **equality** by using **equals()**. You can obtain the **length** of a string by calling the **length()** method. You can obtain the **character** at a **specified index** within a string by calling **charAt()**.

```
1  class StringDemo2 {
2      public static void main(String[] args) {
3          String strOb1 = "First String";
4          String strOb2 = "Second String";
5          String strOb3 = strOb1;
6          System.out.println("Length of strOb1: " + strOb1.length());
7          System.out.println("Char at index 3 in strOb1: " + strOb1.charAt(3));
8          if(strOb1.equals(strOb2))
9              System.out.println("strOb1 == strOb2");
10         else
11             System.out.println("strOb1 != strOb2");
12         if(strOb1.equals(strOb3))
13             System.out.println("strOb1 == strOb3");
14         else
15             System.out.println("strOb1 != strOb3");
16     }
17 }
```

Output:

Length of strOb1: 12
Char at index 3 in strOb1: s
strOb1 != strOb2
strOb1 == strOb3



Using Command-Line Arguments

- Sometimes you will want to pass information into a program when you run it. This is accomplished by passing *command-line arguments* to `main()`.
- A *command-line argument* is the information that directly follows the program's name on the command line when it is executed.
- To access the *command-line arguments* inside a Java program is quite easy—they are stored as strings in a **String** array passed to the `args` parameter of `main()`. The first command-line argument is stored at `args[0]`, the second at `args[1]`, and so on.



Varargs: Variable-Length Arguments

- Modern versions of Java include a feature that simplifies the creation of methods that need to take a **variable number of arguments**. This feature is called **varargs** and it is short for ***variable-length arguments***.
- A **method** that takes a **variable** number of **arguments** is called a ***variable-arity method***, or simply a **varargs method**.
- A variable-length argument is specified by **three periods (...)**. For example:

```
static void vaTest(int ... v) {
```
- This syntax tells the compiler that **vaTest()** can be called with **zero or more arguments**. As a result, **v** is implicitly declared as an **array** of type **int[]**. Thus, inside **vaTest()**, **v** is accessed using the normal **array syntax**.

```
1 class VarArgs {
2     static void vaTest(int ... v) {
3         System.out.print("Number of args: " + v.length + " Contents: ");
4         for(int x : v)
5             System.out.print(x + " ");
6         System.out.println();
7     }
8     public static void main(String[] args) {
9         vaTest(10); // 1 arg
10        vaTest(1, 2, 3); // 3 args
11        vaTest(); // no args
12    }
13 }
```



Varargs: Variable-Length Arguments

- A method can have “normal” parameters along with a **variable-length parameter**. However, the **variable-length parameter must be the last parameter** declared by the method.

```
1 int doIt(int a, int b, double c, int ... vals) {
```

```
1 int doIt(int a, int b, double c, int ... vals, boolean stopFlag) { // Error!
```

- There is one more **restriction** to be aware of: there **must be only one varargs parameter**.

```
1 int doIt(int a, int b, double c, int ... vals, double ... morevals) { // Error!
```



Overloading Vararg Methods

```
1  class VarArgs3 {
2      static void vaTest(int ... v) {
3          ...
4      }
5      static void vaTest(boolean ... v) {
6          ...
7      }
8      static void vaTest(String msg, int ... v) {
9          ...
10     }
11     public static void main(String[] args)
12     {
13         vaTest(1, 2, 3);
14         vaTest("Testing: ", 10, 20);
15         vaTest(true, false, false);
16     }
17 }
```



Varargs and Ambiguity

- The following **overloaded** versions of `vaTest( )` are **inherently ambiguous** even though one takes a normal parameter:

```
1 static void vaTest(int ... v) { // ...  
2 static void vaTest(int n, int ... v) { // ...
```

- Although the parameter lists of `vaTest( )` differ, there is **no way** for the compiler to **resolve** the following call:
`vaTest(1)`
- Does this translate into a call to `vaTest(int ...)`, with one varargs argument, or into a call to `vaTest(int, int ...)` with no varargs arguments? There is no way for the compiler to answer this question. Thus, the situation is **ambiguous**.

```
1  class VarArgs4 {
2      static void vaTest(int ... v) {
3          ...
4      }
5      static void vaTest(boolean ... v) {
6          ...
7      }
8      public static void main(String[] args) {
9          vaTest(1, 2, 3); // OK
10         vaTest(true, false, false); // OK
11         vaTest(); // Error: Ambiguous!
12     }
13 }
```



Local Variable Type Inference with Reference Types

- Recall that when using **local variable type inference**, the type of the variable is specified as **var** and the **variable** must be **initialized**. Earlier examples have shown type **inference** with **primitive types**, but it can also be used with **reference types**. In fact, **type inference** with **reference types** constitutes a **primary use**. Here is a simple **example** that declares a **String** variable called **myStr**:

```
var myStr = "This is a string";
```

- The reason for this is that many **class** types in Java have rather **long names**. For example, in Chapter 13, you will learn about the **FileInputStream** class, which is used to open a file for input operations. In the past, you would declare and initialize a **FileInputStream** using a traditional declaration like the one shown here:

```
FileInputStream fin = new FileInputStream("test.txt");
```

- With the use of **var**, it can now be written like this:

```
var fin = new FileInputStream("test.txt");
```

```
1  class MyClass {
2      private int i;
3      MyClass(int k) { i = k;}
4      int geti() { return i; }
5      void seti(int k) { if(k >= 0) i = k; }
6  }
7  class RefVarDemo {
8      public static void main(String[] args) {
9          var mc = new MyClass(10); // Notice the use of var here.
10         System.out.println("Value of i in mc is " + mc.geti());
11         mc.seti(19);
12         System.out.println("Value of i in mc is now " + mc.geti());
13     }
14 }
```